



SERIES #1: CRACK MNC JAVA INTERVIEW

About Series #1: Crack MNC Java Interview

Who Is It For:

- This series is designed for freshers, graduates, and professionals with 1-5 years of experience.
- It focuses on FAQ questions relevant to major MNC interviews.

How to Use This Guide:

- Use this to learn how real-time interviews ask questions 🎯.
- Focus on applying concepts and showcasing your project experience 💻🚀.

Copyright Notice: 📜

🔴 **Personal Use Only:** This content is strictly for personal use.

🚫 **No Unauthorized Sharing:** Do not post on social media (LinkedIn, Instagram, etc.) without permission.

⚖️ **Strict Copyright Action:** If you share this ePDF on your personal Instagram or LinkedIn without permission, **your account may be suspended or permanently removed** due to **copyright strikes** ⚠️.

👉 **Share Responsibly:** You can share it privately with friends preparing for interviews, but not on social media 🚫

Contact for Permissions:

- Email us at: support@javatechcommunity.com
- DM us on Instagram: [@java_tech_community](https://www.instagram.com/java_tech_community)

DAY#1: Java OOPs Interview Guide

1. OOP Principles Overview

- 📌 Core OOP Concept - Explain w.r.to Your Current Project.
- 💻 Code Snippets - Demonstrating OOP Principles for Interview.

2. Polymorphism

- 📌 What is Polymorphism?
- 🔧 Explain w.r.to Your Current Project.
- 📖 Rules for Overloading and Overriding.
- 🔄 Types: Static vs. Dynamic Polymorphism.
- 📝 Syntax for Overloading and Overriding.
- 💻 Interview Code Snippets:
 - Example 1: Simple Method Overloading.
 - Example 2: Method Overriding in Inheritance.
 - Example 3: Real-World Use Case of Polymorphism in Your Project.
- 📖 Method Overriding Restrictions (Case Study).

DAY#1: Java OOPs Interview Guide

3. Abstract Class

- 📌 Concepts of Abstract Class.
 - 🔧 Explain w.r.to Your Current Project.
 - 📖 Abstract Class Rules.
 - 💻 Interview Code Snippets - Example of Abstract Class.
-

4. Interface

- 📌 Concepts of Interface.
- 🔧 Explain w.r.to Your Current Project.
- 📖 Interface Rules and Best Practices.
- 🚀 Evolution of Interface (Java 8, Java 9).
- 💻 Interview Code Snippets - Example of Interface Usage.

DAY#1: Java OOPs Interview Guide

1. Explain OOPS concepts in detail

Polymorphism

Definition:

- Polymorphism means one object shows different behaviors at different states of its life cycle. ✨

How it is Useful in Real-Time Projects:

- If we apply the polymorphism principle while developing our application, we can use the same object or method for different purposes or operations. ⚙️

Advantages:

- 📁 Saves memory
- 🚀 Increases the performance of the application
- 📖 Increases readability

Inheritance

Definition:

- Inheritance means one class acquires the properties (methods and fields) of another class to achieve reusability of code. ✨

How it is Useful in Real-Time Projects:

- By using the extends keyword, we can achieve inheritance, enabling efficient code reuse and better modularity in applications. ⚙️

Advantages:

- 🔄 Code reusability
- 🔧 Enhancements
- ✅ Software extendability

Abstraction

Definition:

- Abstraction** is a mechanism of hiding the implementation of an object and showing only the essential information to the end user. ✨

How it is Useful in Real-Time Projects:

- We can enhance **security** by hiding the implementation of objects and provide clarity by exposing only essential functionalities. ⚙️

Advantages:

- 🔒 Achieves security by hiding implementation details
- ☀️ Provides essential information to users, reducing complexity
- 📁 Used in **99% of Java applications** to manage application design effectively

Encapsulation

Definition:

- Encapsulation is a programming practice that binds the members of a class together and protects them from unauthorized access. ✨

How it is Useful in Real-Time Projects:

- Encapsulation ensures data protection by restricting access to the members of a class, allowing only controlled interactions. ⚙️

Key Points:

- 🔒 Protects members of a class
- 📦 Binds data and methods together within a class
- ✅ By default, Java supports encapsulation, as members cannot be accessed outside the class



DAY#1: Java OOPs Interview Guide

2. Polymorphism related interview FAQ

Definition:

- Polymorphism means one object shows different behaviors at different states of its life cycle. ✨

How it is Useful in Real-Time Projects:

- If we apply the polymorphism principle while developing our application, we can use the same object or method for different purposes or operations. ⚙️

Example:

- 📌 Method Overloading
- 📌 Method Overriding

Advantages of Polymorphism for our application if we apply Polymorphism concept :

- 📁 Saves memory
- 📖 Increases readability
- 🚀 Improves the performance of the application

3. Types of Polymorphism or [Static v/s dynamic polymorphism]

There are two types of polymorphism

Compile time polymorphism

- Method declaration is bound to the method body at compile time by the compiler.
- Also called **early binding or static binding**.
Example: Method Overloading.
- We can achieve compile-time polymorphism using Method Overloading.

Runtime polymorphism

- Method declaration is bound to the method body at runtime by the JVM.
- Also called **late binding or dynamic binding**.

Example: Method Overriding.

- To achieve runtime polymorphism, we should follow **three mechanisms**
 1. IS-A relationship
 2. Overriding
 3. Class type casting

DAY#1: Java OOPs Interview Guide

4. Method Overloading v/s Method overriding

Method Overloading	Method overriding
Definition If a class has multiple methods with the same name but differing in the number of parameters, it is known as method overloading.	Definition Method overriding is the process of inheriting methods from the superclass and changing their implementation in the subclass.
How to Achieve Method Overloading: Method overloading can be achieved in two ways: - Changing the number of arguments - Changing the data type of arguments	How to Achieve Method Overriding: To achieve method overriding, the following rules must be followed: IS-A relationship: A mandatory relationship must exist between the superclass and subclass. Method Signature: The subclass must retain the method signature of the superclass. Access Level Permission: The subclass can increase the access level but cannot decrease it. Implementation: - Only the subclass object can provide the overridden implementation.
Rules of method overloading: - We can't achieve method overloading by changing return type of only. - It causes ambiguity during execution and through error.	Method Overriding Rules: Private method: Cannot be overridden because it is not visible in the subclass. Static method: Does not support method overriding as it does not support inheritance. Final method: Cannot be overridden as its implementation cannot be changed in the subclass.

DAY#1: Java OOPs Interview Guide

5. What is Abstraction (Abstraction concepts FAQ)

- ✨ Abstraction is one of the OOP principles.
- 🔒 It hides the implementation of an object and shows only essential information to the end user.
- 🛡 Security is achieved by hiding the implementation of objects.
- 📦 99% of Java applications use abstraction to manage complexity.
- 🚫 The implementation is not exposed outside the application.

6. How to Achieve Abstraction

Abstraction can be achieved in **two ways**:

- 📄 Using Abstract Class (0–100%)
- 💻 Using Interface (100%)

7. Steps to Achieve Abstraction Using Interface:

1. 🖋 Create an **interface**.
2. 📄 Declare all the required methods in the **interface body**.
3. 🛠 Create an implementation **class**.
4. The implementation class **must provide implementations** for all interface methods.
5. ✅ **Use the implementation** class object to access the methods.

8. Is Abstraction and Abstract Class the Same?

❌ No, they are different concepts

- ✨ **Abstraction** is an OOP principle that involves hiding the implementation and showing only essential information.
- 💡 **Abstract class** is a type of class declared using the abstract keyword and is one of the ways to achieve abstraction.

DAY#1: Java OOPs Interview Guide

9. Abstract Class and Its Rules

- 📌 It is declared using the abstract keyword.
- 🛠 It can have both abstract and non-abstract methods.
- 📝 Abstract methods must be implemented in the subclass.

It can include:

- 🛠 Constructors
- 📄 Static methods
- ✅ Final methods (but their implementation cannot be changed in the subclass).
- ❌ Objects cannot be created directly from an abstract class.
- 🔴 If a subclass contains unimplemented methods, the subclass must also be declared using the abstract keyword until all methods are implemented.

10. Interface

- 📌 Interface is similar to a class but serves as a definition block.
- 📄 Variables and methods can be declared inside the interface body.

🔑 By default:

- Variables are public, static, final.
- Methods are public, abstract.

Why Do We Need an Interface?

- ✨ To achieve abstraction.
- 🔗 To achieve loose coupling.
- 🌐 To enable multiple inheritance in Java with the help of interfaces.



Series #1: Crack MNC JAVA INTERVIEW in 25 Days in 2025

DAY 1/25: JAN 6th - 2025

join us on javatechcommunity.com for more ePDFs

Task for readers:

These are the most FAQ OOP basics! 🚀

- 📖 **Explore More:** Dive deeper into OOP FAQs, practice, and try small code snippets for each concept 📖. We'll share detailed code snippets on the community channel soon. **For now, we want you to explore and learn!** 🚀
- 🛠️ **For Experienced Professionals:** Reflect on how you've implemented these concepts in your current project.
- 🎓 **For College Students & Graduates:** Relate these concepts to your syllabus and try implementing small projects.

Pro Tip: Think Big, Start Small! 🌟 Let's grow together!

Note:

- This document is meant for brush-ups key concepts & Top FAQ.
- We always recommend working on projects and practicing hands-on coding to solidify your concepts. 📖🚀

Copyright Notice: 📜

🚫 **Personal Use Only:** This content is strictly for personal use.

🚫 **No Unauthorized Sharing:** Do not post on social media (LinkedIn, Instagram, etc.) without permission.

⚖️ **Strict Copyright Action:** If you share this ePDF on your personal Instagram or LinkedIn without permission, your account may be suspended or permanently removed due to copyright strikes ⚠️.

💖 **Share Responsibly:** You can share it privately with friends preparing for interviews, but not on social media 🚫

✉️ **Contact for Permissions:**

- Email us at: support@javatechcommunity.com
- DM us on Instagram: [@java_tech_community](https://www.instagram.com/java_tech_community)